

MindSpore Transformers 大模型系列课程

MindSpore Transformers Safetensors 权重详解

主讲人: CreatY

昇思 MindSpore AI 工程师

MindSpore Transformers SIG Contributor

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

《MindSpore Transformers DeepSeek-R1蒸馏实践》

目录

1. safetensors 权重介绍

2. 权重转换的介绍

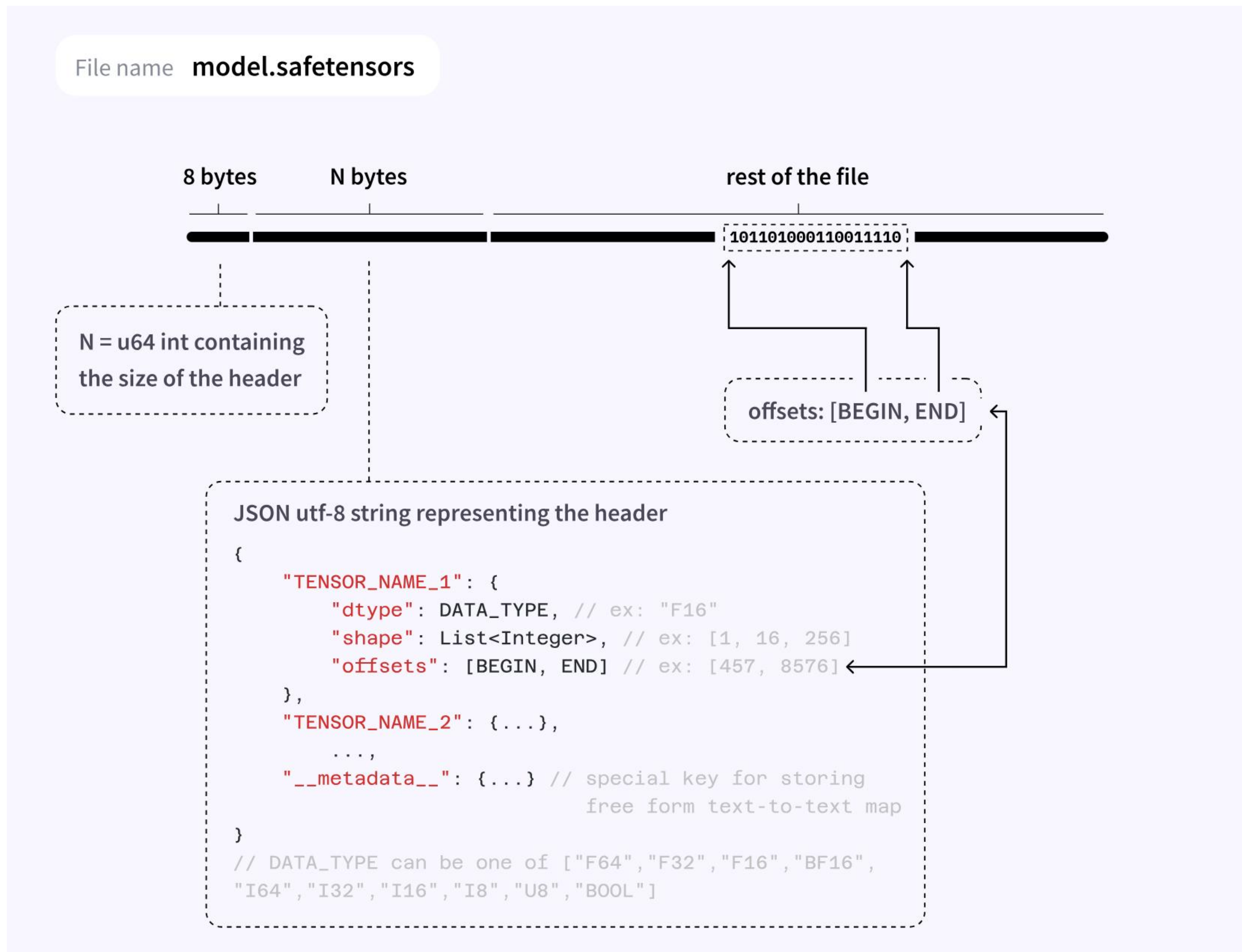
3. 权重切分合并的介绍

4. 实践环节

1. safetensors 权重介绍

safetensors 权重介绍

safetensors 文件结构示意图



safetensors 是一种专为深度学习设计的高性能张量存储格式，具有以下特点：

◆ 安全高效：

避免了传统格式（如pickle）可能存在的安全风险，同时提供了高效的读写性能；

◆ 跨框架兼容：

支持在不同深度学习框架间无缝交换张量数据；

MindSpore 2.6.0 起支持，[MindSpore Transformers 1.5.0 同步接入支持](#)，且 MindSpore Transformers 预计今年年底之前日落 [ckpt 格式](#)。

◆ 零拷贝加载：

可直接映射到内存，无需额外拷贝操作；

◆ 元数据支持：

能够存储张量的元数据（如形状shape、数据类型dtype等）；



MindSpore Transformers 训练使用 safetensors 流程概览

① 加载权重

随机初始化权重

加载MindSpore Safetensors

加载Hugging Face Safetensors

② 模型训练

预训练

继续训练

微调

③ 保存权重

检查点保存

训练结束保存

④ 后续使用

自动切分合并

继续训练、后训练

离线合并

推理、部署、评测

离线合并
权重反转

支持Hugging Face生态使用

MindSpore Transformers 训练中 safetensors 的加载与保存

➤ 训练时加载和保存 safetensors 配置

在使用 MindSpore Transformers 进行模型训练时，需要通过 yaml 对任务相关参数进行配置，下图介绍和展示[加载权重](#)和[保存权重](#)的相关配置项及含义。

```
1 load_checkpoint: '/workspace/Qwen3-0.6B/' # 加载权重路径
2 load_ckpt_format: 'safetensors' # 加载权重格式
3 auto_trans_ckpt: True # 如果需要自动切分，则开启
4 pretrained_model_dir: "/workspace/Qwen3-0.6B/" # 本地 Hugging Face 模型
  文件夹路径
5 remove_redundancy: True # 是否去冗余加载
```

加载权重 yaml 配置示例

```
1 callbacks:
2   - type: CheckpointMonitor # 保存权重配置项
3     prefix: "qwen3" # 权重文件前缀
4     save_checkpoint_steps: 10 # 间隔保存步数
5     keep_checkpoint_max: 3 # 最大保存权重个数
6     async_save: False # 是否开启异步保存
7     checkpoint_format: "safetensors" # 保存权重格式
8     remove_redundancy: True # 是否去冗余保存
```

保存权重 yaml 配置示例

2. 权重转换的介绍与实践

Hugging Face 权重转 MindSpore Transformers 权重

① 加载权重

随机初始化权重

加载MindSpore Safetensors

加载Hugging Face Safetensors

② 模型训练

预训练

继续训练

微调

③ 保存权重

检查点保存 训练结束保存

自动切分合并

离线合并

离线合并
权重反转

④ 后续使用

继续训练、后训练

推理、部署、评测

支持Hugging Face生态使用

Hugging Face 权重转 MindSpore Transformers 权重

➤ 离线转换 Hugging Face 权重

对于规格较大的模型，为了方便使用，MindSpore Transformers 推荐对其进行离线转换后再加载训练的操作，通过库上对应模型的离线转换脚本，对 Hugging Face 权重进行离线转换。现支持模型：[DeepSeek-V3](#)。

```
1  for layer_id in range(total_num_layers):
2      # Dequant weights firstly
3      hf_layer_weights = dequant_layer_weights(layer_id, hf_layer_weights)
4      # Get the replaced weight name and corresponding value in ms
5      ms_layer_weights = defaultdict()
6      # pre/post-process of the DeepSeekV3 model weight
7      if layer_id == 0:
8          ms_layer_weights.update(_model_preprocess_hf_to_ms(hf_layer_weights))
9      if layer_id == total_num_layers - 1:
10         ms_layer_weights.update(
11             _model_postprocess_hf_to_ms(hf_layer_weights, has_mtp_layers)
12         )
13     # MTP Layers process
14     if layer_id > num_layers - 1:
15         ms_layer_weights.update(_mtp_hf_to_ms(layer_id, hf_layer_weights, config))
16     # no MTP layers process
17     else:
18         # MLA Layers process
19         ms_layer_weights.update(
20             _trans_model_layer_norm_hf_to_ms(hf_layer_weights, layer_id)
21         )
22         ms_layer_weights.update(
23             _trans_model_layer_attn_hf_to_ms(hf_layer_weights, layer_id)
24         )
25     # MLP Layers process
26     ms_layer_weights.update(
27         _trans_model_layer_mlp_hf_to_ms(hf_layer_weights, config, layer_id)
28     )
```

脚本代码节选



MindSpore Transformers 权重

Hugging Face 权重转 MindSpore Transformers 权重

➤ 在线加载 Hugging Face 权重

对于规格较小的模型，MindSpore Transformers 支持对其进行在线加载进行微调的操作，通过 yamI 进行配置 `pretrained\_model\_dir` 字段为 Hugging Face 模型文件夹即可在线加载、自动转换 Hugging Face 权重。现支持模型：[Qwen3](#)、[Qwen3-MoE](#)。

```
1 def convert_weight_dict(self, source_dict, **kwargs):
2     ...
3
4     # Concat QKV weight
5     if use_contiguous_weight_layout_attention:
6         self.concat_qkv_weight_infer(wq_keys, wk_keys, wv_keys,
7                                     qkv_dict, condition, ms_weight_dict)
8     else:
9         self.concat_qkv_weight_megatron(
10             wq_keys=wq_keys, wk_keys=wk_keys, wv_keys=wv_keys,
11             qkv_weight_dict=qkv_dict, condition=condition,
12             ms_weight_dict=ms_weight_dict,
13             head_dim=transformer_config.kv_channels,
14             n_kv_heads=transformer_config.num_query_groups,
15             num_attention_heads=transformer_config.num_attention_heads)
16
17     # Concat FFN weight
18     if use_interleaved_weight_layout_mlp:
19         self.concat_ffn_weight_megatron(
20             w1_keys=w1_keys, w3_keys=w3_keys,
21             ffn_weight_dict=qkv_dict, condition=condition,
22             ms_weight_dict=ms_weight_dict,
23             ffn_hidden_size=transformer_config.ffn_hidden_size
24         )
25     else:
26         self.concat_ffn_weight_infer(w1_keys, w3_keys,
27                                     qkv_dict, condition, ms_weight_dict)
28
29     return ms_weight_dict
```

Qwen3 处理逻辑节选



MindSpore Transformers 权重转 Hugging Face 权重

① 加载权重

随机初始化权重

加载MindSpore Safetensors

加载Hugging Face Safetensors

② 模型训练

预训练

继续训练

微调

③ 保存权重

检查点保存

训练结束保存

自动切分合并

离线合并

离线合并
权重反转

④ 后续使用

继续训练、后训练

推理、部署、评测

支持Hugging Face生态使用

MindSpore Transformers 权重转 Hugging Face 权重

➤ 离线反转为 Hugging Face 权重

MindSpore Transformers 提供了各模型对应的离线转换脚本，用户可以在训练结束后，参考相应的使用文档，进行权重反转，反转后的权重支持 vllm 和 Hugging Face 生态的后续操作，亦可用于 MindSpore Transformers 推理任务等。现支持模型：[DeepSeekV3](#)、[Qwen3](#)、[Qwen3-MoE](#)。

MindSpore Transformers权重

处理 Hugging
Face 权重

key、value 转换映射

linear_qkv权重拆分为
linear_q、k、v

linear_fc1 权重拆分为
gate、up_proj

MoE 专家拆分

保存为 Hugging Face 权重

反转脚本处理逻辑示意

MindSpore Transformers ckpt 转 safetensors

➤ ckpt 权重转 safetensors 权重

MindSpore Transformers 较老的版本中使用的权重格式为 ckpt 格式，接入 safetensors 后，为了兼容旧版本训练的权重，MindSpore 提供了 [ckpt 转 safetensors 权重](#) 的接口，同时也有 [safetensors 转 ckpt](#) 的接口。

同时，预计在 2025 年底，MindSpore Transformers 将日落 ckpt 格式。

```
1 import mindspore as ms
2 ms.ckpt_to_safetensors(
3     file_path="./ckpt_save_path/rank0/checkpoint_0.ckpt",
4     save_path="./output/safetensors_path/"
5 )
6
7 # 参数说明
8 # file_path (str) - 包含 checkpoint 文件的目录路径或单个 checkpoint 文件 (.ckpt) 的路径
9 # save_path (str, 可选) - 保存 safetensors 文件的目录路径。默认值: None
10
```

接口使用示例

```
/workspace/mindformers/output/checkpoint/rank_0# ll
4096 Oct 13 16:25 ./
4096 Oct 13 16:19 ../
  87 Oct 13 16:23 meta.json
3938 Oct 13 16:23 qwen3_rank_0-10_1.ckpt
4216 Oct 13 16:23 qwen3_rank_0-graph.meta
```

```
>>> import mindspore as ms
/root/miniconda3/envs/mindspore2.7/lib/python3.10/site-packages/numpy/core/getlimits.py:5
  setattr(self, word, getattr(machar, word).flat[0])
/root/miniconda3/envs/mindspore2.7/lib/python3.10/site-packages/numpy/core/getlimits.py:8
  return self._float_to_str(self.smallest_subnormal)
/root/miniconda3/envs/mindspore2.7/lib/python3.10/site-packages/numpy/core/getlimits.py:5
  setattr(self, word, getattr(machar, word).flat[0])
/root/miniconda3/envs/mindspore2.7/lib/python3.10/site-packages/numpy/core/getlimits.py:8
  return self._float_to_str(self.smallest_subnormal)
>>> ms.ckpt_to_safetensors(file_path="./qwen3_rank_0-10_1.ckpt", save_path="./trans_sf")
>>> exit(0)
```

```
/workspace/mindformers/output/checkpoint/rank_0# ll
4096 Oct 13 16:26 ./
4096 Oct 13 16:19 ../
  87 Oct 13 16:23 meta.json
3938 Oct 13 16:23 qwen3_rank_0-10_1.ckpt
4216 Oct 13 16:23 qwen3_rank_0-graph.meta
4096 Oct 13 16:27 trans_sf/
/workspace/mindformers/output/checkpoint/rank_0# ll trans_sf/
096 Oct 13 16:27 ./
096 Oct 13 16:26 ../
160 Oct 13 16:27 qwen3_rank_0-10_1.safetensors
```

转换操作示例

3.权重切分合并的介绍与实践

MindSpore Transformers 权重自动切分

① 加载权重

随机初始化权重

加载MindSpore Safetensors

加载Hugging Face Safetensors

② 模型训练

预训练

继续训练

微调

③ 保存权重

检查点保存

训练结束保存

④ 后续使用

自动切分合并

继续训练、后训练

推理、部署、评测

离线合并

离线合并
权重反转

支持Hugging Face生态使用

MindSpore Transformers 权重自动切分

➤ MindSpore Transformers 加载权重进行自动切分

MindSpore Transformers 可以对不同策略的权重进行在线自动切分，用户如果在更换策略进行训练时，可以将原始分布式权重合并后进行加载，亦或保留原始策略文件，由 MindSpore Transformers [自动进行合并后在线切分](#)，进行后续任务。

```
def load_checkpoint_with_safetensors(config, model, network, input_data, do_eval=False,
                                     do_predict=False, optimizer=None):
    ...
    if ckpt_file_mode == CheckpointFileMode.MULTI_CHECKPOINT_FILE.value:
        load_checkpoint_files = glob(
            os.path.join(load_checkpoint, f"*.{config.load_ckpt_format}"))
        load_checkpoint_files.sort()
        config.remove_redundancy = False
    elif ckpt_file_mode == CheckpointFileMode.MULTI_CHECKPOINT_FILE_WITH_RANK_ID.value:
        if config.auto_trans_ckpt:
            src_strategy_path = get_merged_src_strategy_path(config)
            unified_safetensors_path = os.path.join(config.output_dir, 'unified_checkpoint/')
            load_checkpoint, file_suffix = _get_src_file_suffix(config)
            unify_safetensors(load_checkpoint,
                              src_strategy_path,
                              unified_safetensors_path,
                              use_parallel=config.use_parallel,
                              file_suffix=file_suffix,
                              remove_redundancy=config.get('remove_redundancy', False))
            load_checkpoint = unified_safetensors_path
            load_checkpoint_files = glob(
                os.path.join(load_checkpoint, f"*.{config.load_ckpt_format}"))
            load_checkpoint_files.sort()
        else:
            ...
    ...
    load_safetensors_checkpoint(config, load_checkpoint_files, network,
                               strategy_path, load_checkpoint, optimizer)
```

自动判断是否需要合并

```
def load_safetensors_checkpoint(config, load_checkpoint_files, network,
                                strategy_path, load_ckpt_path, optimizer):
    """load checkpoint into net."""
    origin_network, _ = _get_origin_network(network)
    if config.use_parallel and config.auto_trans_ckpt:
        logger.info(".....Start load distributed checkpoint to model.....")
        addition_args = {}
        # convert HF name map directly with ms 2.6.0 version
        if check_safetensors_addition_param_support():
            ...
        ms.load_distributed_checkpoint(
            network=network,
            predict_strategy=strategy_path,
            unified_safetensors_dir=load_ckpt_path,
            format=config.load_ckpt_format,
            **addition_args
        )
        # Load optimizer param in resume_training
        hyper_param_file = os.path.join(load_ckpt_path, 'hyper_param.safetensors')
        if optimizer and config.resume_training:
            if not os.path.exists(hyper_param_file):
                raise FileNotFoundError(rf"No hyper_param.safetensors in given dir: {load_ckpt_path}")
            logger.info(".....Start load hyper param into optimizer.....")
            hyper_param_dict = ms.load_checkpoint(ckpt_file_name=hyper_param_file,
                                                  format='safetensors')
            update_global_step(config, hyper_param_dict)
            ms.load_param_into_net(optimizer, hyper_param_dict)
```

加载safetensors

MindSpore Transformers 权重离线合并

① 加载权重

② 模型训练

③ 保存权重

④ 后续使用

随机初始化权重

预训练

自动切分合并

继续训练、后训练

加载MindSpore Safetensors

自动切分

继续训练

检查点保存

训练结束保存

离线合并

推理、部署、评测

加载Hugging Face Safetensors

离线转换
在线加载

微调

离线合并
权重反转

支持Hugging Face生态使用

MindSpore Transformers 权重离线合并

➤ MindSpore Transformers 分布式权重进行离线合并

MindSpore Transformers 提供了对训练下来的分布式权重离线合并的[功能脚本](#)，[使用该脚本](#)可以将多卡分布式权重合并为单卡完整权重，方便后续更换策略续训或进行评测、权重反转等任务。

```
1 python toolkit/safetensors/unified_safetensors.py \  
2   --src_strategy_dirs ./output/strategy/ \  
3   --mindspore_ckpt_dir ./output/checkpoint/ \  
4   --output_dir ./merged_safetensors \  
5   --file_suffix "10_1" \  
6   --has_redundancy False \  
7   --filter_out_param_prefix "adam_" \  
8   --max_process_num 32
```

脚本使用示例

```
INFO - [2025-10-13 16:48:45 args config:]  
INFO - file_suffix=10_1  
INFO - filter_out_param_prefix=adam_  
INFO - format=safetensors  
INFO - has_redundancy=False  
INFO - max_process_num=32  
INFO - mindspore_ckpt_dir=./output/checkpoint/  
INFO - output_dir=./merged_safetensors/10_1_ckpt_convert  
INFO - src_strategy_dirs=./output/strategy/  
INFO - [2025-10-13 16:48:45 Task start...]  
INFO - [2025-10-13 16:48:45 Start merge strategy]  
INFO - [2025-10-13 16:48:45 Merge strategy completed]  
INFO - [2025-10-13 16:48:45 Start merge safetensor]  
INFO - [2025-10-13 16:48:50 Merge safetensor completed]  
INFO - merged_safetensor path: ./merged_safetensors/10_1_ckpt_convert/unified_safe  
INFO - [2025-10-13 16:48:50 Task completed time:]  
INFO - [2025-10-13 16:48:50 Task total cost time: 5.515031814575195s]
```

脚本使用日志

```
/mindformers# ll merged_safetensors/10_1_ckpt_convert/unified_safe/  
  
16:48 ./   
16:48 ../   
16:48 hyper_param.safetensors   
16:48 param_name_map.json   
16:48 part0.safetensors   
/mindformers#
```

合并文件示例

MindSpore Transformers 合并后权重使用场景

➤ 合并后训练权重进行继续训练/后训练

MindSpore Transformers 可以加载合并后的权重进行[续训](#)，配置 yaml 中的 `auto\_trans\_ckpt: True` 即可，此时 MindSpore Transformers 会对合并权重进行自动切分。

详情可关注后续《MindSpore Transformers 断点续训介绍与实践》课程。

➤ 合并后训练权重进行推理评测

MindSpore Transformers 推理不仅支持加载 Hugging Face 权重进行推理，还支持[使用训练合并后的权重进行推理](#)，对训练推理网络做了自动适配，如 qkv 离散排布转连续排布、网络 key、value 转换（linear_q_down、linear_kv_down 自动合并为 linear_qkv_down 等）。

详情可关注后续《MindSpore Transformers 推理介绍与实践》等课程。

➤ 反转合并后训练权重为 Hugging Face 权重

使用模型对应的离线反转脚本，将合并后的权重反转为 Hugging Face 权重，不仅可以复用 Hugging Face 生态，还可用于 MindSpore Transformers 推理、vllm 推理、评测部署等。

4. 实践环节

实践流程概览

① 加载权重

随机初始化权重

加载MindSpore Safetensors

加载Hugging Face Safetensors

② 模型训练

预训练

继续训练

微调

③ 保存权重

检查点保存 训练结束保存

自动切分合并

离线合并

离线合并
权重反转

④ 后续使用

继续训练、后训练

推理、部署、评测

支持Hugging Face生态使用

SIG组织

昇思社区

昇腾社区

魔乐社区

SIG > MindSpore Transformers

MindSpore Transformers

https://gitee.com/mindspore/community/tree/master/signs/mindspore_transformers

MindSpore Transformers SIG (Special Interest Group) 是 MindSpore 开源社区下聚焦 Transformer 类大模型全流程开发与优化的技术团队，涵盖大模型的预训练、微调、推理与部署等关键环节。SIG 团队致力于提升 Transformer 类大模型在性能与易用性方面的表现，推动其在实际场景中的高效应用与落地。

核心成员(6)

Maintainers (2)

L

Lin-Bert

He Qinglin

S

suhaibo

Su Haibo

Committers (4)

C

chenrayray

Chen Xinrui

H

hss-shuai

Huang Shengshuai

R

renyujin

Ren Yujin

Z

zyw-hw

zhang Youwen

会议与活动

工作会议

查看会议纪要 >

会议时间：每月一次，北京时间周四 16:00 - 17:00 以公开线上会议形式讨论SIG组议题。

会议前3-5天会在邮件列表中发起sig组相关议题收集，并发出会议时间和会议链接。

邮件

dev@mindspore.cn 订阅邮件

最新日程：2025/09/04

2025/09

04

2025/08

07

2025/07

03

MindSpore Transformers SIG月例会

2025-09-04 16:00 - 17:00

会议详情：-

发起人：chenrayray

会议时间：16:00-17:00

会议平台：tencent

会议ID：952843996

会议链接：<https://meeting.tencent.com/dm/Fig5yhhFz7E0>

Etherpad链接：<https://etherpad.mindspore.cn/p/sig-MindSpore-Transformers-meetings>



课程交流群



MindSpore Transformers SIG群



MindSpore Transformers 内测文档



MindSpore Transformers 开源仓库



谢谢观看！

MindSpore Transformers Safetensors 权重详解

MindSpore Transformers 大模型系列课程